

Reviewer Pack — Minimal Norm of Non-Arithmetic Curves Bounds Resolution Proo...

Entry dfc25d60540a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 00:41:19 UTC

Minimal Norm of Non-Arithmetic Curves Bounds Resolution Proof Width

Entry ID: dfc25d60540a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-30 00:41:19 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Non-Arithmetic Curves) **Field B** (complexity object): Boolean Function Complexity: Resolution Proof Complexity

Statement:

For every satisfiable Boolean formula φ with n variables, there exists a non-arithmetic curve C associated with φ such that the width of the resolution proof tree for φ is $O(n^{2/|C|}(1/4))$.

Rationale (proposer's reasoning):

Non-Arithmetic Curves provide a novel way to encode the complexity of Boolean formulas. Their inherent properties could reveal hidden structures in the resolution proof process, potentially leading to new insights into proving lower bounds.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 7a956cae5d136720

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all satisfiable Boolean formulas φ with n variables, the minimal norm $|C|$ of a non-arithmetic curve associated with φ satisfies $|C|^{1/4} \leq n^2/\text{width_of_resolution_proof_tree}$ AND $\text{width_of_resolution_proof_tree} \leq 10n^2$ for at least 95% of seeds. The conjecture is falsified if any seed produces a resolution proof tree width exceeding $10n^2$ or if $|C|^{1/4} > n^2/\text{width_of_resolution_proof_tree}$ for more than 5% of seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	HITS	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "non-arithmetic curves" AND "resolution proof complexity" - "Algebraic Geometry" AND "Boolean function complexity" - "Minimal Norm" AND resolution proof width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        formula = []
        for i in range(1, n + 1):
            formula.append(f"X{i}")
        return " & ".join(formula)

    def resolution_width(phi):
        # Placeholder function to simulate the width of a resolution proof tree
        # This is a dummy implementation and should be replaced with actual logic
        return len(phi.split(" & ")) ** 2

    def non_arithmetic_curve_norm(n):
        # Placeholder function to compute the minimal norm of a non-arithmetic curve
        # This is a dummy implementation and should be replaced with actual logic
        return n ** 0.5

    n = random.randint(5, 40)
    phi = generate_formula(n)
    width = resolution_width(phi)
    C_norm = non_arithmetic_curve_norm(n)

    if width > 10 * n**2:
        return {
            "metric_name": "resolution_width",
            "metric_value": width,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
```

```

        "counterexample": f"Width exceeds  $10n^2$  for  $n=\{n\}$ "
    }

    if C_norm**(1/4) > n**2 / width:
        return {
            "metric_name": "resolution_width",
            "metric_value": width,
            "instances_tested": 1,
            "n_max": n,
            "conjecture_holds": False,
            "counterexample": f" $C\_norm^{(1/4)} > n^2/width$  for  $n=\{n\}$ "
        }

    return {
        "metric_name": "resolution_width",
        "metric_value": width,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31))
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    std_width = math.sqrt(sum((r["metric_value"] - mean_width)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\" $C\_norm^{(1/4)} > n^2/width$ \" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

IAL: {'metric_name': 'resolution_width', 'metric_value': 625, 'instances_tested': 1, 'n_max': 25, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 529, 'instances_tested': 1, 'n_max': 23, '
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 81, 'instances_tested': 1, 'n_max': 9, 'co
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 1444, 'instances_tested': 1, 'n_max': 38,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 256, 'instances_tested': 1, 'n_max': 16, '
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 1296, 'instances_tested': 1, 'n_max': 36,
TRIAL: {'metric_name': 'resolution_width', 'metric_value': 361, 'instances_tested': 1, 'n_max': 19, '

```

```

TRIAL: {'metric_name': 'resolution_width', 'metric_value': 289, 'instances_tested': 1, 'n_max': 17, '
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7063c128.py", line 89, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7063c128.py", line 89, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^
KeyErr

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results show that for multiple seeds, the minimal norm of a non-arithmetic curve associated with the Boolean formulas does not satisfy the co | next: Investigate further to identify why the resolution proof tree widths exceed the expected bounds and explore alternative methods or conditions that could lead to a valid proof.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13786
2	preregistration	ollama_remote	glm4:latest	0	0	10571
3	novelty	ollama_remote	glm4:latest	0	0	8303
4	novelty	ollama_remote	glm4:latest	0	0	8818
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12071
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9175
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8229
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8939
9	judge	ollama_remote	glm4:latest	0	0	14250

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 94141 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/dfc25d60540a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71

2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/dfc25d60540a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/dfc25d60540a.tar.gz` (if generated)